

Praxis: Konfigurierbares Kernelmodul

Parameter ändern, Verhalten beobachten,
Modul integrieren

2.05

Ein Modul, drei kontrollierbare Eigenschaften

01

INTERVALL

100–60.000 ms
uint

02

AUSGABE

ein / aus
bool

03

INTEGRATION

Defaults + Autoload
kmod

Bauen → Laden → Ändern → Prüfen → Installieren

Das Modul sendet einen konfigurierbaren Heartbeat

PARAMETER

Eingaben

- interval_ms
- verbose

KERNEL

Verarbeitung

- Delayed Work
- plant den nächsten Lauf

LOG

Ausgabe

- pr_info() schreibt den Heartbeat

verbose = 1

Meldung bei jedem Lauf

verbose = 0

Worker läuft weiter, Log bleibt ruhig

Projekt und laufenden Kernel zuerst prüfen

TERMINAL

```
$ mkdir -p ~/kernel-training/configurable
$ cd ~/kernel-training/configurable
$ touch configurable.c Makefile

$ uname -r
$ test -d /lib/modules/$(uname -r)/build && echo READY
```

PRÜFPUNKT

READY erscheint und beide Quelldateien liegen im Projektverzeichnis.

01 Testsystem

Mit Snapshot oder anderer Wiederherstellungsmöglichkeit arbeiten.

Nur auf einem geeigneten Testsystem mit Wiederherstellungsmöglichkeit arbeiten.

Der Lebenszyklus bleibt bewusst minimal

CONFIGURABLE.C

```
#include <linux/init.h>
#include <linux/module.h>

static int __init configurable_init(void)
{
    pr_info("configurable: loaded\n");
    return 0;
}

static void __exit configurable_exit(void)
{
    pr_info("configurable: removed\n");
}

module_init(configurable_init);
module_exit(configurable_exit);
MODULE_LICENSE("GPL");
```

01 init

Laden beobachten
und Fehler melden.

02 exit

Arbeit stoppen und
Ressourcen freigeben.

03 Regel

Jeder erfolgreiche init-Schritt
braucht Cleanup.

Zwei Variablen öffnen die Konfiguration

PARAMETER-DEFINITION

```
static unsigned int interval_ms = 1000;
static bool verbose = true;

module_param(interval_ms, uint, 0644);
MODULE_PARM_DESC(interval_ms, "Heartbeat interval in ms");

module_param(verbose, bool, 0644);
MODULE_PARM_DESC(verbose, "Enable heartbeat messages");
```

INTERVAL_MS

Default: 1000
Typ: uint
Zur Laufzeit schreibbar

VERBOSE

Default: true
Typ: bool
Zur Laufzeit schreibbar

Metadaten dokumentieren Typ, Default und Laufzeitberechtigung.

Kbuild erzeugt und beschreibt das Modul

MAKEFILE

```
obj-m += configurable.o

KDIR := /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

TERMINAL

```
$ make

$ file configurable.ko

$ modinfo configurable.ko

$ modinfo -p configurable.ko
```

ERGEBNIS

- configurable.ko
- zwei dokumentierte Parameter

Bei Fehlern zuerst Kernelversion, Header und Makefile-Tabs prüfen.

Startwerte werden an der Modulgrenze gesetzt

TERMINAL

```
$ sudo insmod ./configurable.ko interval_ms=500 verbose=1  
$ sudo dmesg | tail -n 10  
$ cat /sys/module/configurable/parameters/interval_ms  
$ cat /sys/module/configurable/parameters/verbose
```

01 configurable: loaded

Initialisierung im Kernel-Log

02 500

Wirksames Intervall

03 Y

Boolean-Parameter ist aktiv

Die Werte unter sysfs bestätigen die wirksame Konfiguration.

Delayed Work erzeugt den Heartbeat

CONFIGURABLE.C

```
#include <linux/workqueue.h>
#include <linux/jiffies.h>

static void heartbeat(struct work_struct *work);
static DECLARE_DELAYED_WORK(heartbeat_work, heartbeat);

static void heartbeat(struct work_struct *work)
{
    unsigned int ms = READ_ONCE(interval_ms);
    if (READ_ONCE(verbose))
    {
        pr_info("configurable: heartbeat %u ms\n", ms);
    }

    schedule_delayed_work(&heartbeat_work, msecs_to_jiffies(ms));
}
```

01 READ_ONCE

Skalaren Wert pro Lauf einmal lesen.

02 msecs_to_jiffies

Millisekunden in Kernel-Zeiteinheiten umrechnen.

03 Nächster Lauf

Neue Werte gelten bei der nächsten Planung.

Start und Stopp müssen symmetrisch bleiben

INIT / EXIT

```
static int __init configurable_init(void)
{
    pr_info("configurable: loaded\n");
    schedule_delayed_work(&heartbeat_work, 0);
    return 0;
}

static void __exit configurable_exit(void)
{
    cancel_delayed_work_sync(&heartbeat_work);
    pr_info("configurable: removed\n");
}
```

START

nach erfolgreichem Setup



STOP

vor dem letzten Log



SICHER

keine ausstehende Arbeit

sysfs ändert das Verhalten ohne Neuladen

TERMINAL

```
$ sudo dmesg -w  
$ echo 250 | sudo tee /sys/module/configurable/parameters/interval_ms  
$ echo 0 | sudo tee /sys/module/configurable/parameters/verbose  
$ echo 1 | sudo tee /sys/module/configurable/parameters/verbose
```

BEOBACHTUNG

- schnellerer Takt
- Ruhe
- Meldungen kehren zurück

GRENZE

Der bereits geplante Lauf wird nicht rückwirkend verschoben.

Laufzeitänderungen greifen kontrolliert ab der nächsten Planung.

Der Setter begrenzt das Intervall

SETTER

```
static int set_interval(const char *val,
                       const struct kernel_param
*kp)
{
    unsigned int value = 0;
    int ret = kstrtouint(val, 0, &value);
    if (ret)
    {
        return ret;
    }
    if (value < 100 || value > 60000)
    {
        return -EINVAL;
    }
    return param_set_uint(val, kp);
}
```

PARAMETER-OPS

```
static const struct kernel_param_ops ops = {
    .set = set_interval,
    .get = param_get_uint,
};

module_param_cb( interval_ms, &ops,
&interval_ms, 0644);
```

Eine kleine Matrix deckt die Risiken ab

START	<code>interval_ms=500 verbose=1</code>	Lädt · Heartbeat sichtbar
GRENZE	<code>interval_ms=99</code>	Abgelehnt · Wert unverändert
LAUFZEIT	<code>500 → 200 → 1000</code>	Takt folgt beim nächsten Lauf
STUMM	<code>verbose: 1 → 0 → 1</code>	Log stoppt und startet
ENTLADEN	<code>rmmod während Heartbeat</code>	Kein Log nach removed

Bestanden, wenn alle Ergebnisse reproduzierbar sind.

modprobe übernimmt Installation und Defaults

INSTALLATION

```
$ sudo install -D -m 0644 configurable.ko /lib/modules/$(uname -r)/extra/configurable.ko
$ sudo depmod -a
$ sudo modprobe configurable
$ sudo modprobe -r configurable
```

/etc/modprobe.d/configurable.d

```
options configurable interval_ms=500 verbose=1
```

/etc/modules-load.d/configurable.conf

```
configurable
```

Für frühe Bootphasen können Distributionswerkzeuge und ein initramfs-Update nötig sein.

Das Modul ist steuerbar und überprüfbar

01

Robustheit

Zähler für
ausgelassene Läufe

02

Diagnose

pr_debug() +
Dynamic Debug

03

Konfiguration

Weiteren bool- oder
uint-Wert ergänzen

Build ✓ Parameter ✓ Laufzeitänderung ✓ Logs ✓ modprobe ✓

Konfigurieren. Beobachten. Sicher aufräumen.